

Description of the Machine-Learning- and Graph-Based Pattern-Mining Methods

Machine-Learning-Based Pattern-Detection Method

This study employs a machine-learning approach based on Support Vector Machines (SVM) because their effectiveness has been demonstrated in related research (Ramamoorthi, 2020). The approach comprises four principal phases: data preparation, feature-vector construction, SVM-model training, and pattern prediction. Algorithm 1 presents the SVM procedure used to identify microservice patterns.

Algorithm 1 SVM-based detection of microservice patterns.

Require: *PBMS*: Pattern-Based Microservice System outputs, *SelectedPatterns*: set of patterns to detect

Ensure: *DetectedPatterns*: set of identified microservice design patterns

```

1: DetectedPatterns  $\leftarrow \emptyset$ 
                                     ▷ Phase 1: Data preparation
2: for each pattern P in SelectedPatterns do
3:   Extract microservices  $MS = \{ms_1, ms_2, \dots\}$  from P
4:   Assign roles to microservices: Master, Worker, Client
5: end for
                                     ▷ Phase 2: Feature-vector construction
6: for each microservice ms in MS do
7:   for each role r in {Master, Worker, Client} do
8:     Initialize feature vector  $FV_r \leftarrow []$ 
9:     Compute InEdge by multiplying the primes corresponding to
10:      supported operations (Get:2, Post:3, Put:5, Delete:7)
11:
12:     Compute OutEdge analogously
13:     for each operation op and target microservice  $ms_t$  do
14:       Compute interaction features; for example, M-get→W = 1
15:       if the Master calls the Worker via Get, and 0 otherwise
16:
17:     end for
18:     Compute the remaining eight features defined in Fig. 9
19:     Append  $FV_r$  to  $FV_{ms}$ 
20:   end for
21:   Combine role-specific vectors into the final  $FV_{ms}$  (length = 30)
22:   Add  $FV_{ms}$  to dataset D
23: end for
                                     ▷ Phase 3: SVM-model training
24: Train the SVM classifier  $SVM_{model}$  on dataset D with pattern labels
                                     ▷ Phase 4: Pattern prediction
25: for each new microservice instance  $ms_{new}$  do
26:   Construct  $FV_{ms_{new}}$  using the procedure above
27:    $pattern \leftarrow SVM_{model}.predict(FV_{ms_{new}})$ 
28:    $DetectedPatterns \leftarrow DetectedPatterns \cup \{pattern\}$ 
29: end for
30: return DetectedPatterns

```

• **Phase 1 (Data Preparation):** The microservices corresponding to each pattern are first extracted from the systems generated by the proposed benchmark. Each microservice is assigned one of three principal roles: *Master*, *Worker*, or *Client*. Master microservices perform management and coordination tasks, Worker microservices implement the main business operations and core functionality, and Clients submit service requests. Each role is represented by ten elementary features, as shown in Figure 1. The complete feature vector therefore contains $3 \times 10 = 30$ values. Role assignment provides the basis for defining the behavioral characteristics of each microservice and constructing the corresponding feature vectors.

• **Phase 2 (Feature-Vector Construction):** A 30-dimensional feature vector is constructed for each microservice to quantify its behavior. The following paragraphs briefly explain its elements. The first two features associated with each role in Figure 1 are InEdge and OutEdge.

InEdge indicates which operations (GET, POST, PUT, and DELETE) a microservice supports when responding to requests. A prime number is assigned to each operation: GET:2, POST:3, PUT:5, and DELETE:7. When a microservice supports several operations, its InEdge value is the product of the corresponding primes. For example, if a Master microservice supports GET and POST and can be invoked through both operations, its InEdge value is $2 \times 3 = 6$.

OutEdge identifies the operations through which a microservice interacts with other microservices and is encoded in the same manner as InEdge. Eight additional features specify the target role with which the microservice interacts through each operation. For example, M-get→W equals one when the Master invokes GET on a Worker and equals zero otherwise. The remaining vector features shown in Figure 1 can be quantified for each pattern in the same way.



Fig. 1: Feature vector of the microservices in the patterns used by the SVM.

Finally, the role-specific vectors are concatenated to form the complete 30-dimensional feature vector for each microservice. This construction process allows the SVM to use data generated by PAMBS to identify both the individual operations of each microservice and its interaction patterns across the system.

- **Phase 3 (SVM-Model Training):** The constructed feature vectors form the dataset used to train the SVM classifier. Each sample is assigned the label of its corresponding design pattern. The SVM learns decision boundaries that separate the patterns in the 30-dimensional feature space. It was selected because of its ability to handle high-dimensional data and its demonstrated effectiveness in related pattern-detection studies.

- **Phase 4 (Pattern Prediction):** For each new microservice instance, a feature vector is computed using the procedure described in Phase 2. The trained SVM model then predicts the corresponding design pattern, and all identified patterns are collected in the final set of detected patterns. This procedure enables automatic identification of microservice design patterns from their actual behavior and supports systematic analysis and comparative evaluation of pattern implementations in microservice-based systems.

Graph-Based Pattern-Detection Method

The graph-based pattern-matching algorithm identifies subgraphs in an input graph $G = (V, E)$ that match a set of predefined patterns. It comprises three principal phases: node labeling, edge filtering, and pattern matching through Dual Simulation, together with auxiliary checks that enforce global constraints. Algorithm 2 presents the graph-based procedure used to identify microservice patterns.

- **Phase 1 (Node Labeling):** Each node $v \in V$ in the input graph is evaluated against the constraints of each pattern $P \in \text{Patterns}$. The objective is to determine which node types defined by the pattern can be assigned to each graph node. An initially empty label set $t(v)$ is created for every node. The algorithm iterates over all node types specified by the pattern and tests whether the node satisfies the corresponding constraints. These constraints may include the node label, node degree, operational properties such as HTTP methods or append-only behavior, and other pattern-specific characteristics. Each node is then labeled with every type whose constraints it satisfies. By ensuring that only relevant nodes proceed to edge filtering and pattern matching, this phase reduces unnecessary computation.

- **Phase 2 (Edge Filtering):** After the nodes have been labeled, the graph edges are filtered according to the constraints of the current pattern. For each edge $e \in E$, the algorithm determines whether the edge satisfies the required conditions, which may include its type, associated metadata, operation type, or other pattern-specific properties. Only edges satisfying these criteria are retained in the filtered subgraph G_{filtered} . Eliminating invalid edges substantially reduces the search space and confines subsequent computation to edges that may belong to a valid pattern-matching subgraph.

- **Phase 3 (Pattern Matching):** The algorithm iterates over each node w in the filtered graph G_{filtered} . For each node, it extracts a subgraph called a *ball*, which contains all nodes within the radius defined by the pattern diameter. A method based on Dual Simulation is then applied to identify the set of nodes S_w within the ball that may match the pattern. This set is used to extract candidate subgraphs G_s whose structures match the pattern. Each candidate is validated against optional global constraints, such as quorum rules, append-only restrictions, or leader-node uniqueness. Only candidates satisfying these constraints are added to the final set of matched subgraphs. This phase constitutes the core of the algorithm and combines local and global constraints to improve detection accuracy.

- **Auxiliary Functions:** Three auxiliary functions support the main algorithm. $\text{NodeConstraintsSatisfied}(v, \text{nodeType})$ determines whether a node satisfies the constraints associated with a node type by considering properties such as its label, degree, and operations. $\text{EdgeConstraintsSatisfied}(e, P)$ determines whether an edge satisfies the requirements of a specific pattern, including its type, metadata, and operations. Finally, $\text{GlobalConstraintsSatisfied}(G_s, P)$ validates candidate subgraphs against optional global rules to ensure compliance with structural or behavioral requirements such as quorum rules, append-only properties, or leader uniqueness.

The algorithm identifies pattern-matching subgraphs by progressively narrowing the search space through node labeling, edge filtering, and targeted pattern matching. Combining local constraints on nodes and edges with global validation provides both high accuracy and computational efficiency when detecting design patterns in complex microservice-system graphs.

Algorithm 2 General pattern-matching algorithm.

Require: Graph $G = (V, E)$, set of patterns Patterns

Ensure: Set of matched subgraphs MatchedSubgraphs

```

1: MatchedSubgraphs  $\leftarrow \emptyset$ 
2: for each pattern  $P$  in Patterns do ▷ Phase 1: Node labeling
3:   for each node  $v$  in  $V$  do
4:      $t(v) \leftarrow \emptyset$ 
5:     for each  $nodeType$  in  $P.NodeTypes$  do
6:       if NodeConstraintsSatisfied( $v, nodeType$ ) then
7:          $t(v) \leftarrow t(v) \cup \{nodeType\}$ 
8:       end if
9:     end for
10:  end for ▷ Phase 2: Edge filtering
11: ValidEdges  $\leftarrow \emptyset$ 
12: for each edge  $e$  in  $E$  do
13:   if EdgeConstraintsSatisfied( $e, P$ ) then
14:     ValidEdges  $\leftarrow$  ValidEdges  $\cup \{e\}$ 
15:   end if
16: end for
17:  $G_{\text{filtered}} \leftarrow \text{Subgraph}(G, \text{ValidEdges})$  ▷ Phase 3: Graph matching
18: for each node  $w$  in Nodes( $G_{\text{filtered}}$ ) do
19:    $ball \leftarrow \text{ExtractBall}(G_{\text{filtered}}, w, P.Diameter)$ 
20:    $S_w \leftarrow \text{DualSim}(P, ball)$ 
21:    $G_s \leftarrow \text{ExtractMatchedSubgraph}(P, ball, S_w)$ 
22:   if  $G_s \neq \text{nil}$  and GlobalConstraintsSatisfied( $G_s, P$ ) then
23:     MatchedSubgraphs  $\leftarrow$  MatchedSubgraphs  $\cup \{G_s\}$ 
24:   end if
25: end for
26: end for
27: return MatchedSubgraphs
28: function NODECONSTRAINTSatisfied( $v, nodeType$ )
29:   return Boolean ▷ Check node label, degree, and properties
30: end function
31: function EDGECONSTRAINTSatisfied( $e, P$ )
32:   return Boolean ▷ Check edge type, metamodel, function, and operation
33: end function
34: function GLOBALCONSTRAINTSatisfied( $G_s, P$ )
35:   return Boolean ▷ Check quorum, leader uniqueness, and related constraints
36: end function

```

References

Ramamoorthi, V. (2020). Machine learning models for anomaly detection in microservices. *Quarterly Journal of Emerging Technologies and Innovations*, 5(1), 41–56.